

PRÁCTICA 10

Seguridad Móvil en Android

Análisis, Reversing y Mitigación de Vulnerabilidades

SACZN Cybersecurity

InsecureBankv2 · APKTool · JADX-GUI · Kali Linux

1. INTRODUCCIÓN

El presente informe detalla el proceso de ingeniería inversa aplicado a dispositivos móviles Android, utilizando como objetivo la aplicación vulnerable **InsecureBankv2**. El objetivo técnico es comprender el funcionamiento interno de una APK mediante técnicas de decompilación y análisis estático, identificando vulnerabilidades presentes en su diseño y proponiendo estrategias de mitigación para asegurar la integridad de los datos y la privacidad del usuario.

¿Qué es el Reversing de Aplicaciones Android?

Una APK (*Android Package*) es un archivo comprimido que contiene todos los componentes de una aplicación: bytecode Dalvik (**.dex**), recursos gráficos, el manifiesto de permisos (**AndroidManifest.xml**), bibliotecas nativas y metadatos de firma digital.

La ingeniería inversa de APKs consiste en desensamblar o decompilar estos componentes para recuperar una representación legible del código original. Esto permite a los investigadores de seguridad:

- Auditar la lógica de autenticación y gestión de credenciales.
- Detectar secretos embebidos (claves de API, contraseñas codificadas en el código fuente).
- Identificar comunicaciones inseguras o ausencia de cifrado.
- Verificar que no se almacene información sensible en texto plano.
- Evaluar la robustez del código frente a ataques de manipulación.

2. DESARROLLO

2.1 Decompilación con APKTool

APKTool es la herramienta estándar para el desensamblado de aplicaciones Android. Permite decodificar los recursos binarios de una APK, reconstruir el **AndroidManifest.xml** en formato legible y extraer el bytecode Smali para su inspección directa.

Se ejecutó el siguiente comando sobre la APK objetivo:

```
apktool d InsecureBankv2.apk -o InsecureBank_ficheros
```

El proceso completó exitosamente las siguientes etapas: carga de la tabla de recursos, decodificación del *AndroidManifest.xml*, baksmaling del archivo *classes.dex* y copia de assets, bibliotecas y archivos originales.

```
(kali@kali)-[/media/sf_practica_10]
$ apktool d InsecureBankv2.apk -o InsecureBank_ficheros
I: Using Apktool 2.7.0-dirty on InsecureBankv2.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/kali/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

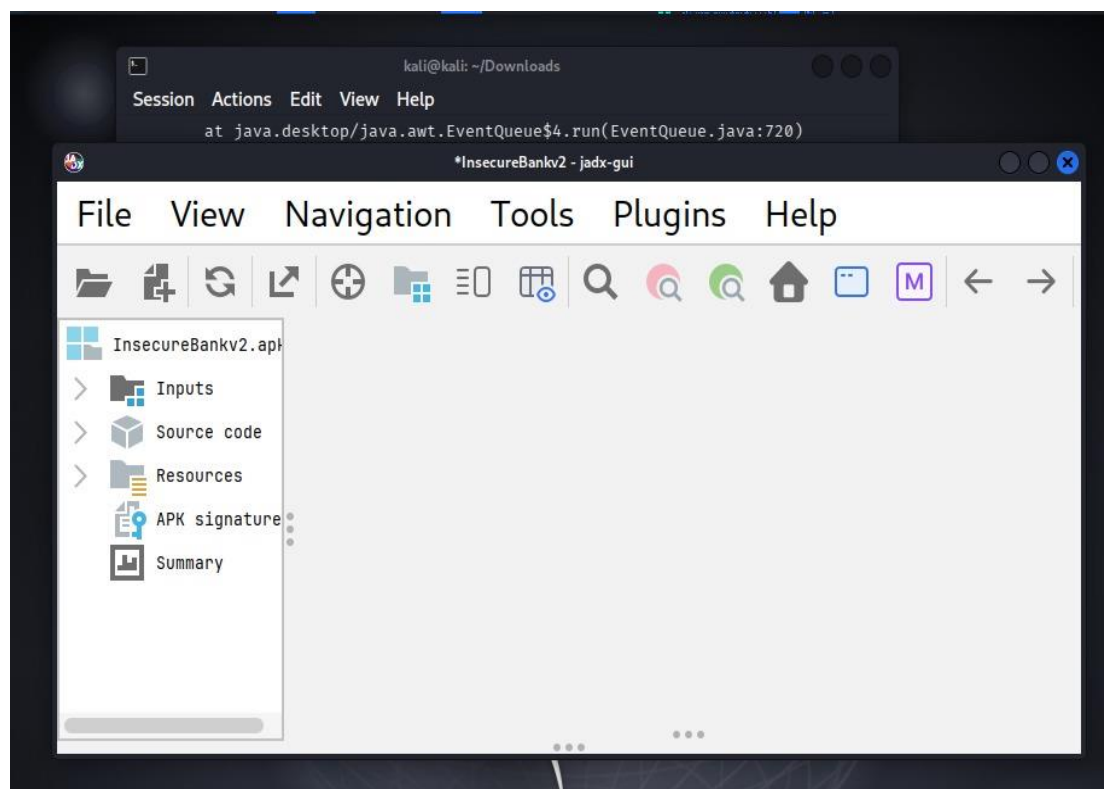
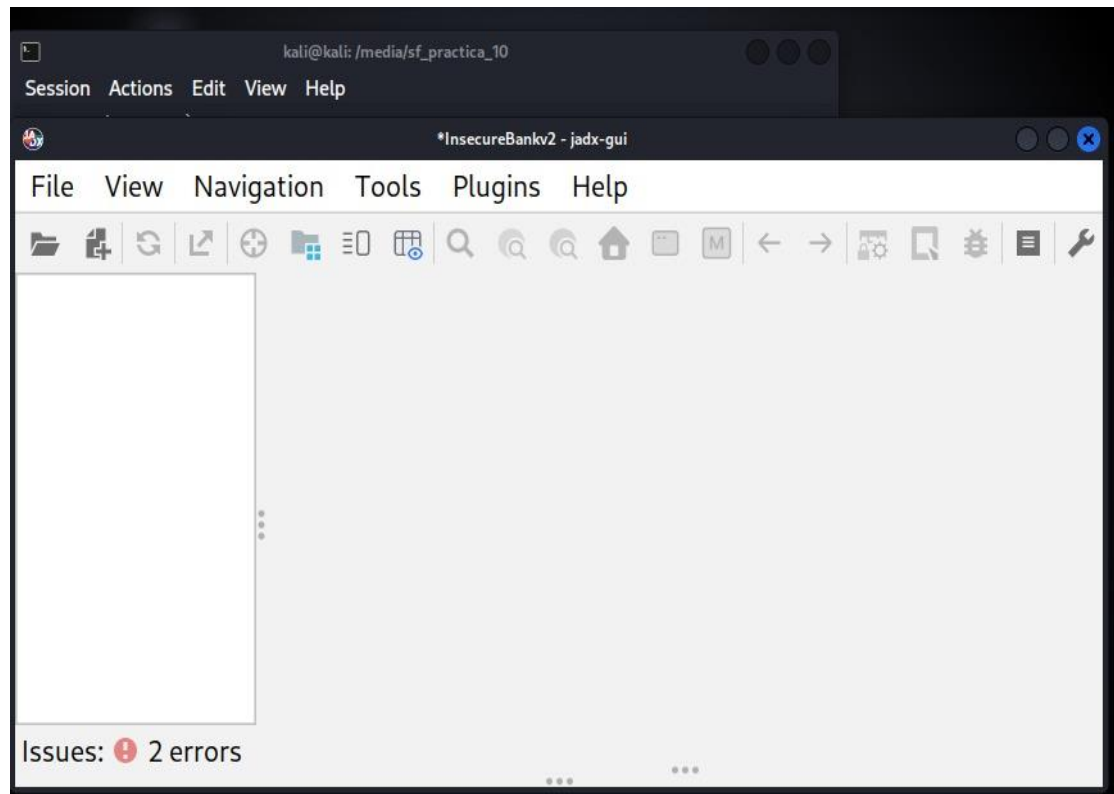
(kali@kali)-[/media/sf_practica_10]
$
```

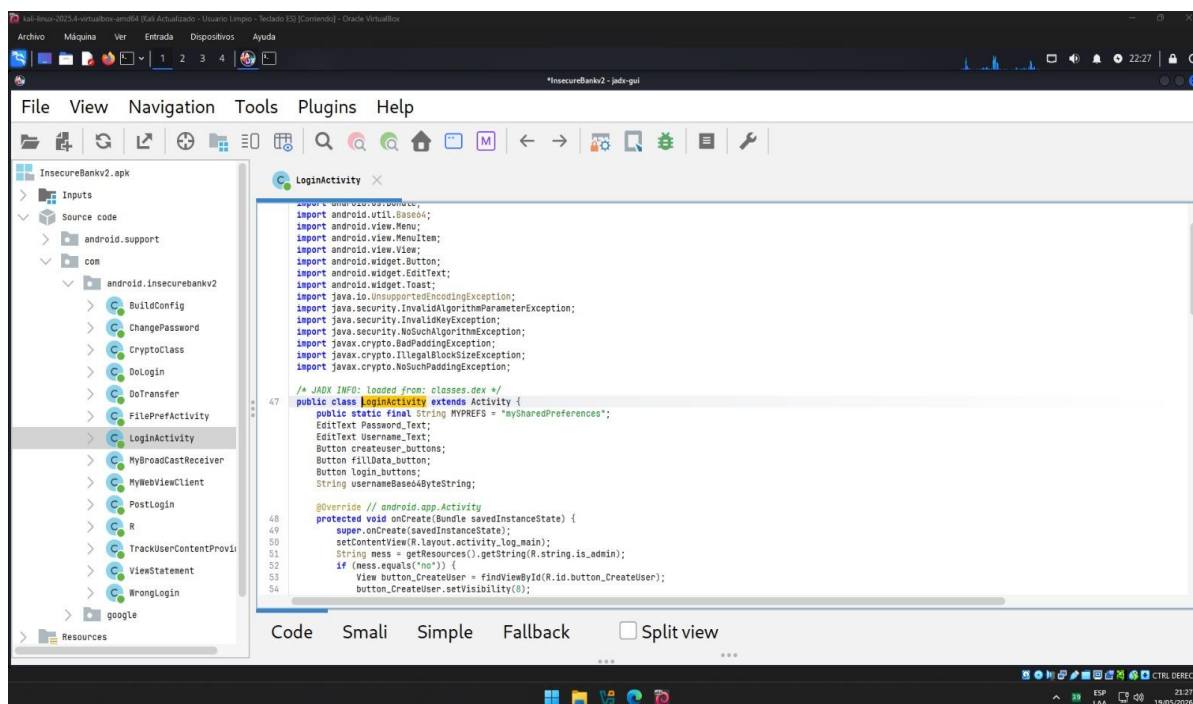
2.2 Análisis Estático con JADX-GUI

JADX-GUI es actualmente la herramienta más avanzada y completa para el análisis estático de aplicaciones Android. A diferencia del flujo tradicional *Dex2Jar* + *JD-GUI* —que requiere dos pasos separados y presenta limitaciones con bytecode moderno—, JADX integra todo el proceso en un único entorno: convierte directamente el bytecode *.dex* a código fuente Java de alta fidelidad, con soporte para clases lambda, generics y código generado por compiladores recientes. Por esta razón, JADX-GUI ha reemplazado de facto al flujo *Dex2Jar* + *JD-GUI* en el trabajo profesional de análisis de APKs.

Una vez cargada la APK se obtuvo la estructura completa del paquete **com.android.insecurebankv2**, incluyendo clases críticas como:

- **LoginActivity**
- **ChangePassword**
- **DoTransfer**
- **CryptoClass**
- **PostLogin**
- **FilePrefActivity**
- **WrongLogin**





Del análisis de **LoginActivity** se identificaron los siguientes hallazgos:

- **Credenciales en SharedPreferences:** La constante `MYPREFS = "mySharedPreferences"` expone el nombre del archivo donde se almacenan datos del usuario, legible en dispositivos con acceso root.
- **Ausencia de ofuscación:** Nombres de clases, métodos y campos completamente legibles (`usernameBase64ByteString`, `fillData_button`), facilitando la comprensión inmediata de la lógica de negocio.
- **Imports criptográficos visibles:** La importación de `javax.crypto.*` y `android.util.Base64` permite auditar en detalle la implementación criptográfica.
- **Control de flujo expuesto:** La lógica condicional basada en el string `is_admin` es directamente visible, lo que podría permitir la manipulación de privilegios.

2.3 Medidas de Protección y Mitigación

Tras la identificación de las superficies de exposición, se proponen las siguientes recomendaciones técnicas:

- **Ofuscación con R8 / ProGuard:** Renombra clases, métodos y campos a identificadores no significativos, incrementando la complejidad del análisis y reduciendo el tamaño del APK.
- **Gestión segura de secretos:** Claves y credenciales nunca deben estar embebidas en el código. Deben obtenerse en tiempo de ejecución o almacenarse mediante Android Keystore System, respaldado por hardware.
- **Cifrado de datos locales:** Toda información sensible en SharedPreferences o SQLite debe cifrarse con AES-256-GCM. EncryptedSharedPreferences de Jetpack Security simplifica esta implementación.

- **Certificate Pinning:** Ancla el certificado del servidor para evitar ataques de interceptación (MitM), independientemente de las autoridades de certificación del sistema.
- **Runtime Application Self-Protection (RASP):** Comprobaciones en tiempo de ejecución para detectar root, emuladores, Frida/Xposed o modificaciones del APK, respondiendo con cierre de sesión o desactivación de funciones críticas.
- **Play Integrity API:** Verifica que la aplicación se ejecuta en un dispositivo Android no comprometido y con la APK original, mediante la API de Google Play Integrity.

3. CONCLUSIONES

- Se ha validado la eficacia de la ingeniería inversa como metodología de auditoría, demostrando que herramientas como APKTool y JADX-GUI permiten recuperar lógica de negocio crítica en minutos.
- El análisis de **InsecureBankv2** evidencia que la ausencia de ofuscación y la gestión inadecuada de datos sensibles exponen completamente la arquitectura interna a cualquier auditor.
- JADX-GUI se consolida como la herramienta de referencia actual para análisis estático de APKs, superando al flujo Dex2Jar + JD-GUI en fidelidad y soporte de bytecode moderno.
- La seguridad de una aplicación móvil debe basarse en un modelo de **defensa en profundidad**, combinando ofuscación, cifrado de datos, protección de credenciales y RASP.